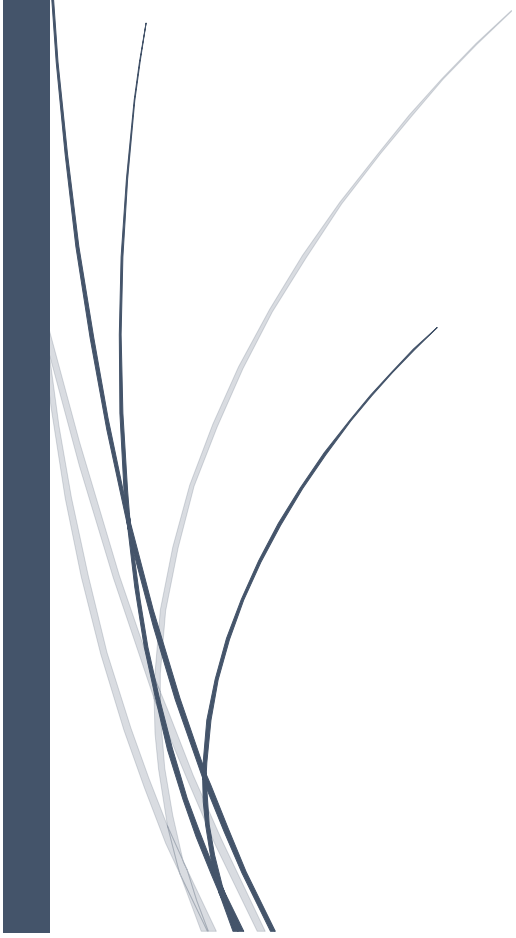
@VANZA

UT6. Persistencia de Datos en Ficheros

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. 2
2. Ficheros de texto plano 2
2.1. Trabajar con ficheros sin estructura. 2
2.2. Ficheros de ancho fijo. 5
2.3. Trabajar con ficheros en formato CSV y TSV 8
2.4. Trabajar con ficheros XML 14
2.5. Trabajar con ficheros JSON. 18

1. Introducción.

El principal propósito que tiene un fichero que contiene datos es almacenar la información en disco hasta el momento que el programa se inicialice y se almacene en memoria o sea requerida a medida que se necesite. Como no podía ser de otra forma, Python permite trabajar con diferentes tipos de ficheros de datos y las librerías del núcleo cuentan con diferentes formatos, además de las de terceros.

2. Ficheros de texto plano

Como ya sabemos, los ficheros de texto plano permiten que cualquier persona o programa que tenga acceso al fichero puede descubrir el contenido del mismo debido a que no está oculto y la codificación de la secuencia de caracteres es legible por el ser humano y cualquier sistema.

Los podemos categorizar en cuatro grupos:

- **Sin estructura básica**→ no presentan ninguna estructura interna.
- **De ancho fijo**→ presentan un ancho definido para cada campo, asemejándose a una tabla donde el ancho de las columnas está definido por un número de caracteres fijo.
- **Tabulados**→ son ficheros que presentan un delimitador definido para todo el contenido. Un ejemplo muy claro son los ficheros "CSV", en los que cada fila está separada por *saltos de línea* y cada columna está separada por la *coma.*
- **Formato estándar**→ la información se representa de forma coordinada entre quien la envía y quien la recibe.

2.1. Trabajar con ficheros sin estructura.

La mejor opción para trabajar con este tipo de ficheros es el uso de la función del núcleo de Python **open().**

- **open(**nombreFichero, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)

  nombreFichero: ruta o nombre del fichero que se quiere abrir.
  mode: define el modo de acceso al fichero.
  'r' - lectura (defecto)
  'w' - escritura, permite escribir en el fichero, truncando el contenido anterior
  'x' - crea un nuevo fichero y lo abre (si ya existe se lanza una excepción)
  'a' – abre el fichero para añadir contenido

'b' – abre el fichero en modo binario, devolviendo el contenido en objetos de tipo bytes

buffering: opcional, se usa para definir la política de buffering

encoding: opcional, define la codificación para abrir el fichero.

e*rrors:* opcional, permite manejar ficheros mal codificados. La opción por defecto es None que elevará una excepción de tipo *ValueError* ante cualquier problema. (strinct, ignore, replace, surrogateescape, xmlcharrefreplace, backslashreplace, namereplace

newline: define cómo analizar los saltos de línea. Por defecto es None, que habilita el modo de salto de línea universal.

Normalmente, los parámetro usados son **file, mode, encoding** y **errors.**

La función open devuelve un objeto de tipo **file** (también denominado descriptor de fichero) que puede ser usado por sí mismo o pasado como argumento a muchas librerías que trabajan con ficheros.

file.read()→ lee todo el contenido del fichero. Si el fichero es muy grande no es recomendable dado que carga todo el contenido en memoria. También se puede establecer un valor en bytes de manera que sólo devuelva ese tamaño

ej. file.read(1024) devuelve una muestra del fichero de 1K

file.readline()→ lee el fichero línea por línea.

file.realines()→ lee todas las líneas del fichero y las almacena en una lista de cadenas. Para procesar las líneas será necesario iterar sobre la lista.

file.write(texto)→ el archivo debe estar abierto en modo escritura ('w', 'a', 'x')

file.close()→ cierra el archivo liberando el descriptor de fichero (file) y asegurando que los cambios se lleven a cabo de forma correcta.

file.seek(offset, whence=0)→mueve el puntero dentro de un archivo abierto en Python.

offset: es el número de bytes que quieres mover

whence: 0(desde el principio, por defecto); 1 (desde la posición actual); 2 (desde el final del fichero)

with … as file:

En Python es una forma segura y conveniente utilizar esta estructura para trabajar con archivos dado que garantiza que se cierre automáticamente al finalizar el bloque de código definido en su interior, incluso si se produce un error.

Vamos a realizar el siguiente ejemplo. Tenemos el fichero de texto **quijote.txt** con el siguiente texto (las líneas numeradas para entenderlo mejor)

1 En un lugar de la Mancha, de cuyo nombre no quiero acordarme,
2 no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero,
3 adarga antigua, rocín flaco y galgo corredor.
4 Una olla de algo más vaca que carnero, salpicón las más noches,
5 duelos y quebrantos los sábados, lentejas los viernes,
6 algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.
7 El resto della concluían sayo de velarte, calzas de velludo
8 para las fiestas, con sus pantuflos de lo mesmo, y los días de entresemana se honraba
9 con su vellorí de lo más fino.
10 Tenía en su casa una ama que pasaba de los cuarenta

Abrimos el fichero para lectura e imprimimos usando readline() y read(). Observa el resultado para que entiendas mejor qué ha pasado.:

```
with open("quijote.txt", "r") as f:
      linea = f.readline()
      texto = f.read()

print(texto)
print(linea)
```

Con readlines():

```
with open("quijote.txt", "r", encoding="utf-8") as f:


      lineas=f.readlines()

for linea in lineas:
      print(linea.strip()) # utilizamos la función strip para eliminar los
      espacios en blanco
      # al principio y al final de la línea y el salto de línea
```

Lo podemos hacer de forma mas eficiente de la siguiente manera:

```
with open("quijote.txt", "r", encoding="utf-8") as f:
    for linea in f: # Itera sobre cada línea del archivo
        print(linea.strip())
```

Ahora que conocemos cómo leer y escribir desde o en un fichero de texto, a partir del fichero **quijote.txt,** vamos a crear otro nuevo que contenga sólo las lineas pares del fichero:

```
with open("quijote.txt", "r") as f:
  with open("quijotepares.txt", "w") as fpares:
    indice = 1
    for linea in f: # Itera sobre cada línea del archivo
        if not indice % 2: # Si el índice es par
            fpares.write(linea)
        indice += 1
  # Cierra los archivos automáticamente al salir del bloque with

# El archivo quijotepares.txt contendrá solo las líneas pares del archivo
quijote.txt
# Como hemos utilizado el bloque with, ahora debemos abrir el archivo para
leerlo y comprobar que se ha creado correctamente
with open("quijotepares.txt", "r") as fpares:
    for linea in fpares:
        print(linea, end="") # Imprime cada línea del archivo
quijotepares.txt sin añadir una nueva línea
```

2.2. Ficheros de ancho fijo.

Como hemos dicho antes, los ficheros de ancho fijo componen los valores internos distribuyéndose por cada línea con una longitud fija, a modo de columnas de una tabla. Al saber con anterioridad la longitud de los campos de cada fila y la posición de cada valor, se pueden consultar tanto verticalmente como horizontalmente para extraer un valor específico.

Veamos mediante el siguiente ejemplo cómo leer los datos de un fichero de ancho fijo con la siguiente estructura:

Empleado (3 caracteres)

Nombre (10 caracteres)

Apellidos (10 caracteres)

Salario (5 caracteres)

Así, tenemos, como ejemplo, el fichero Salarios.txt que tiene las siguientes líneas

```
125Antonio   Garcia    25000
010Marta     Vega      30000
255María     Gómez     40000
189Agustín   Pérez     27500
745Rodrigo   Vicente   28000
154Andrea    Parla     36000
```

Vamos a leer el fichero y mostrar los campos.

```
def leer_fichero_ancho_fijo(ruta):
    empleados = []
    with open(ruta, 'r', encoding='utf-8') as f:
        for linea in f:
            #vamos leyendo cada línea del fichero, extrayendo cada campo
eliminado los espacios en blanco
            empleado = linea[0:3].strip()
            nombre = linea[3:13].strip()
            apellido = linea[13:23].strip()
            salario = int(linea[23:28].strip())
            empleados.append((empleado, nombre, apellido, salario))
    return empleados # la función devuelve una lista de tuplas, cada una
representando un empleado

#ejemplo de uso con el fichero "Salarios.txt"
empleados = leer_fichero_ancho_fijo('Salarios.txt')
for e in empleados:
    print(e)
```

Añadimos una función para escribir (añadir) datos al fichero controlando si el fichero ya existe o no: (Ejemplo completo)

```
# Importamos la clase Path del módulo pathlib para trabajar con rutas de
ficheros de forma más sencilla y eficiente
from pathlib import Path

def escribir_fichero_ancho_fijo(ruta, listEmpleados):
    #este método recibe como parámetros la ruta del fichero y una lista de
empleados
    #cada empleado es una tupla con 4 elementos: (empleado, nombre, apellido,
salario)
    fichero = Path(ruta)# creamos un objeto Path con la ruta del fichero
    #comprobamos si el fichero existe, si existe lo abrimos en modo 'a'
(append)
    # si no existe lo creamos en modo 'w' (write)
```

```
    if fichero.exists():
        modo='a'
    else:
        modo='w'
    with open(ruta, modo, encoding='utf-8') as f:
        for empleado in listEmpleados:
            #escribimos cada empleado en una línea del fichero, rellenando con
espacios en blanco hasta alcanzar el ancho fijo
            f.write(f"{empleado[0]:<3}{empleado[1]:<10}{empleado[2]:<10}{emplea
do[3]:<5}\n")

def leer_fichero_ancho_fijo(ruta):
    emple = []
    with open(ruta, 'r', encoding='utf-8') as f:
        for linea in f:
            #vamos leyendo cada línea del fichero, extrayendo cada campo
eliminado los espacios en blanco
            empleado = linea[0:3].strip()
            nombre = linea[3:13].strip()
            apellido = linea[13:23].strip()
            salario = int(linea[23:28].strip())
            emple.append((empleado, nombre, apellido, salario))
    return emple # la función devuelve una lista de tuplas, cada una
representando un empleado

#ejemplo de uso con el fichero "Salarios.txt"
#
listaEmpleados = [
    ('001', 'Juan', 'Pérez', 18000),
    ('002', 'Ana', 'Gomez', 35000),
    ('003', 'Luis', 'Martinez', 49000),
    ('004', 'María', 'Lopez', 46500),
]

escribir_fichero_ancho_fijo('Salarios.txt', listaEmpleados)

empleados = leer_fichero_ancho_fijo('Salarios.txt')
for e in empleados:
    print(e)
```

Ejercicio.

Antes hemos mencionado que existen librerías de terceros que permiten trabajar con los ficheros. Observa el siguiente ejemplo en que se utiliza la librería **pandas** y toda la potencia y eficiencia que nos ofrece.

```
import pandas as pd
def ancho_fijo_pandas(fichero,anchos):
    return pd.read_fwf(fichero, widths=anchos,header=None, names=["ID",
"Nombre", "Apellido", "Salario"])

#uso
if __name__ == "__main__":
    # Definimos los anchos de cada columna
    anchos = [3, 10, 10, 5]
    # Leemos el fichero con los anchos definidos
    df = ancho_fijo_pandas("Salarios.txt", anchos,)
    # Mostramos el dataframe resultante
    print(df)
```

2.3. Trabajar con ficheros en formato CSV y TSV

El contenido de los ficheros puede estar delimitado por separadores entre los valores. De esta forma se optimiza el espacio frente a otros formatos como el del tamaño fijo. El delimitador puede ser cualquier carácter pero existen dos por excelencia que son las comas y el tabulador. Así, dan nombre a los archivos en formato CSV (comma-separated values) y TSV (tab-separated values) y sus extensiones son .csv y .tsv respectivamente.

Python cuenta con la librería **csv** para trabajar con este tipo de ficheros que permite extraer y escribir valores en separados por un delimitador especial.

- **csv.reader(csvfile, dialect='excel', **fmtparams)**: devuelve un objeto para iterar sobre las líneas del fichero definido por **csvfile.** Parámetros:
  csvfile: suele ser un fichero o un iterador que devuelve cadenas.
  dialect: define le dialecto para escanear el fichero. Suele ser '**excel**' (por defecto) que utiliza ',' como separador. Otro común es '**excel-tab**' que utiliza el tabulador como separador.
  fmtparams: parámetros que se utilizan para sobreescribir propiedades del dialecto seleccionado o incluso te permite no tener que declarar el dialecto

- **csv.writer(csvfile, dialect='excel', **fmtparams):** permite escribir información en el fichero csvfile estableciendo delimitadores. La explicación de los parámetros es idéntica que la de la función **reader.**
- **writer.writerow(linea):** método de un writer que guarda en el fichero los valores del iterable **linea.**
- **writer.writerows(lineas):** método de un writer que usa como parámetro un iterador de iteradores (lista de listas) y guarda cada iterador en el fichero como una nueva linea.

Ejemplo:

Partimos del fichero **Salarios.csv** que es idéntico a los anteriores pero con el delimitador "," entre cada valor.

125,Antonio,Garcia,25000
010,Marta,Vega,30000
255,María,Gómez,40000
189,Agustín,Pérez,27500
745,Rodrigo,Vicente,28000
154,Andrea,Parla,36000
001,Juan,Pérez,18000
002,Ana,Gomez,35000
003,Luis,Martinez,49000
004,María,Lopez,46500

```
import csv
def escribir_csv(fichero_csv):
    #damos por sentado que el fichero existe y lo abrimos en modo append
    # es más que conveniente utilizar newline='' para evitar líneas en blanco
en el csv
    with open(fichero_csv, mode='a',newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['005','Juan','Rivera', 30000])
        writer.writerow(['006','Manuela','Peralta', 40000])
        writer.writerow(['007','Andrea','Solís', 50000])

def leer_csv(fichero_csv):
    with open(fichero_csv, mode='r', encoding='utf-8') as f:
        lineasFichero = csv.reader(f)
        for row in lineasFichero:
            print(row)

# uso de las funciones
escribir_csv('Salarios.csv')
leer_csv('Salarios.csv')
```

- **csv.DictReader(fichero, fieldnames=None, restkey=None, restval=None, dialect='excel', *args, **kwargs):** devuelve un objeto de tipo *reader* que presenta la información como un diccionario. Es más que recomendable cuando el fichero **csv** tiene cabeceras dado que el fichero tomará esa información de la cabecera para establecer los pares clave:valor. Si el fichero no tiene cabeceras, se pueden establecer mediante el parámetro **fieldnames**. Parámetros:
  fichero: flujo de datos o descriptor de fichero con el que se trabaja

fieldnames: secuencia de campos que determinan las claves. Si se omite se tomará la primera fila del fichero para establecer las claves (cuidado).
restkey: si alguna fila tiene más valores que los especificados en fieldnames se guardarán en la lista especificada en este parámetro.
restval: si alguna fila tiene menos valores que los especificados en fieldnames, se añade este valor que por defecto es None.
dialect: especifica el dialecto que se usará para la lectura de los valores del fichero.
args y kwargs: valores extra del dialecto.

Ejemplo: utilizamos el fichero anterior **Salarios.csv** que no tiene línea de cabecera:

```
import csv
def leerCSV_Dict(ruta):
    with open(ruta, mode='r', encoding='utf-8') as fichero:
        #se define el lector de csv utilizando el parametro fieldnames para
definir los nombres de las columnas
        lector = csv.DictReader(fichero,fieldnames=['Código','Nombre',
'Apellido', 'Salario'])
        for fila in lector:
            print(fila)
#usar la función
ruta = 'Salarios.csv'
leerCSV_Dict(ruta)
```

- Prueba a utilizar los parámetros restkey y restval añadiendo algún valor extra en filas del fichero y quitando algún valor en otras…

- **csv.DictWriter(fichero, fieldnames, restval='', extrasaction='raise', dialect='excel', *args, **kwargs):** devuelve un objeto de tipo *writer* que permite escribir la información en el fichero en forma de un diccionario. Parámetros:
  fichero: flujo de datos o descriptor de fichero con el que se trabaja.
  fieldnames: secuencia de campos que determinan las claves que deben escribirse en el fichero.
  restval: si algún diccionario le falta alguna clave de las que están definidas en fieldname, se utilizará esta por defecto.
  extrasaction: permite especificar qué hacer cuando falte alguna de las claves necesarias. Por defecto su valor será *raise* que provocará que se eleve una excepción del tipo *ValueError.*Si se especifica el valor *ignore* se escribirá el valor de *restval.*
  dialect: especifica el dialecto para la lectura de los valores del fichero.
  args y kwargs: valores extra para el dialecto elegido.

Ejemplo con escritura y lectura:

```
import csv
def escribirCSV_Dict(ruta):
    #se define la lista de empleados
    dictSalarios =
[{'Codigo':'001','Nombre':'Juan','Apellido':'Pérez','Salario':18000},
                    {'Codigo':'002','Nombre':'Ana','Apellido':'Gomez','Sal
ario':35000},
                    {'Codigo':'003','Nombre':'Luis','Apellido':'Martinez',
'Salario':49000},
                    {'Codigo':'004','Nombre':'María','Apellido':'Lopez','S
alario':46500}]
    #se abre el fichero en modo escritura y se define el escritor de csv.
    #  que el modo 'w' sobrescribe el fichero si ya existe
with open(ruta, mode='w', encoding='utf-8' ,newline='') as fichero:

        escritor = csv.DictWriter(fichero, fieldnames=['Codigo','Nombre',
'Apellido', 'Salario'])
        #se escriben los datos de los empleados
        for empleado in dictSalarios:
            escritor.writerow(empleado)

def leerCSV_Dict(ruta):
    with open(ruta, mode='r', encoding='utf-8') as fichero:
        #se define el lector de csv utilizando el parametro fieldnames
para definir los nombres de las columnas
        lector = csv.DictReader(fichero,fieldnames=['Código','Nombre',
'Apellido', 'Salario'],restkey='Otros', restval='No definido')
        for fila in lector:
            print(fila)
#usar la función
ruta = 'Salarios.csv'
escribirCSV_Dict(ruta)
#leer el fichero
leerCSV_Dict(ruta)
```

- **DictWriter.writeheader():** método de un DictWriter que permite guardar una fila con las cabeceras del fichero a partir del *fieldnames*
- **Class csv.Sniffer:** es una clase que nos da información del formato del fichero. Dos métodos:

  sniff(sample, delimiter=None)→ analiza el fragmento del archivo definido por ejemplo y que normalmente se lee del propio fichero.

  has_header(sample)→ analiza un fragmento del archivo y devuelve true si tiene cabeceras en la primera fila. Igualmente, *sample* es una muestra leida del propio archivo (fichero.read(1024) por ejemplo)

Ejemplo: Vamos a ver el siguiente ejemplo completo que utiliza una clase para trabajar con los datos de un fichero. Se trata de un fichero que contiene los datos de planetas (Nombre, Masa y Radio) y hace el cálculo de su gravedad…Observa el ejemplo y estudia todas las sentencias para comprenderlo.

planetas.csv

```
Nombre,Masa,Radio
Mercurio,3.303e+23,2.4397e6
Venus,4.869e+24,6.0518e6
Tierra,5.976e+24,6.37814e6
Marte,6.421e+23,3.3972e6
Jupiter,1.9e+27,7.1492e7
Saturno,5.688e+26,6.0268e7
Urano,8.686e+25,2.5559e7
Neptuno,1.024e+26,2.4746e7
```

Código:

ficherodelimitado.py

```
from dataclasses import dataclass
import csv
# definiendo la clase Planeta con el decorador @dataclass para simplificar la
creación de clases
@dataclass
class Planeta:
    nombre: str
    masa: float
    radio: float
    # definimos el método __post_init__ para convertir los atributos a float después
de la inicialización
    # esto es útil si los datos se leen de un archivo y vienen como cadenas
    def __post_init__(self):
        self.radio = float(self.radio)
        self.masa = float(self.masa)
    # propiedad para calcular la gravedad del planeta
    # usando el decorador @property para definir un método como propiedad
    @property
    def gravedad(self):
        g_constante = 6.673e-11 # constante de gravitación universal
        # calculando la gravedad usando la fórmula g = constante * masa / radio al
cuadrado
        return g_constante * self.masa / (self.radio ** 2)


def lectura_de_planetas(nombre_fichero):
    with open(nombre_fichero, 'r') as fr:
```

```
        reader = csv.reader(fr)

        tiene_cabecera = csv.Sniffer().has_header(fr.read(1024))
        fr.seek(0)  # colocando la posición de lectura al inicio de nuevo
        if tiene_cabecera:
            cabecera = next(reader)  # se ignora la cabecera
        for fila in reader:
            yield Planeta(*fila)

# usando la función lectura_de_planetas para leer los datos de un archivo CSV
# y crear instancias de la clase Planeta
for p in lectura_de_planetas('planetas.csv'):
        print(f'La gravedad en {p.nombre} es {p.gravedad}')
```

Trabajar con ficheros delimitados en Python es simple haciendo uso de las clases *DictWriter* y *DictReader.* Pero si hacemos uso de las librería *pandas* que proporciona las funciones **read_csv** y **to_csv** para leer y escribir en un fichero.

Observa el siguiente ejemplo que, a partir del fichero anterior **planetas.csv** genera otro fichero añadiendo un nuevo valor **diametro** (el valor del radio al cuadrado):

```
import pandas as pd

def lectura_y_escritura_usando_pandas(fichero_entrada, fichero_salida):
    df = pd.read_csv(fichero_entrada)
    # Se aplican las modificaciones en el dataframe
    print(df.head())
    df['Diametro'] = df['Radio'] ** 2
    df.to_csv(fichero_salida, index=False)

lectura_y_escritura_usando_pandas('planetas.csv', 'nuevos_planetas_pandas.csv')
```

Este ejemplo tan sencillo es una muestra de la potencia de librerías como **pandas.** En multitud de ocasiones, no será conveniente abusar de este tipo de librerías dado que son muy grandes y su importación consume mucha memoria. Sobre todo cuando los cambios en los datos no son tan complejos…

2.4. Trabajar con ficheros XML

Trabajar con los archivos CSV está muy bien cuando los datos no tienen jerarquías definidas. Para esos casos en los que los datos tienen estructuras jerarquizadas y son más complejos, se usan otros tipos de formatos. Uno de los más populares aún hoy en día es **XML (**de sus siglas Lenguaje de Marcado Extensible).

Este formato tiene la característica de que permite definir jerarquías de forma independiente al sistema en el que se utilice de manera que se pueden intercambiar los datos entre sistemas distintos con el mismo fichero.

XML tiene dos componentes principales, las etiquetas y los atributos. Las etiquetas forman los bloques de datos y pueden o no contener atributos.

Si definimos el fichero anterior de los planetas mediante un fichero XML, podría ser perfectamente así:

planetas.xml

```
<sistemasolar>
   <planeta nombre="Mercurio">
      <masa>3.303e23</masa>
      <radio unidad="km">2.4397e6</radio>
   </planeta>
   <planeta nombre="Venus">
      <masa>4.869e+24</masa>
      <radio unidad="km">6.051e6</radio>
   </planeta>
   <planeta nombre="Tierra">
      <masa>5.976e+24</masa>
      <radio unidad="km">6.37814e6</radio>
   </planeta>
   <planeta nombre="Marte">
      <masa>6.421e+23</masa>
      <radio unidad="km">3.397e6</radio>
   </planeta>
```

```
    <planeta nombre="Jupiter">
        <masa>1.9e+27</masa>
        <radio unidad="km">7.1492e7</radio>
    </planeta>
    <planeta nombre="Saturno">
        <masa>5.688e+26</masa>
        <radio unidad="km">6.0268e7</radio>
    </planeta>
    <planeta nombre="Urano">
        <masa>8.686e+25</masa>
        <radio unidad="km">2.5559e7</radio>
    </planeta>
    <planeta nombre="Neptuno">
        <masa>1.024e+26</masa>
        <radio unidad="km">2.4746e7</radio>
    </planeta>
</sistemasolar>
```

Como se puede comprobar, en este tipo de ficheros la información se muestra de forma clara y concisa, con una jerarquía muy marcada.

La colección se llama *sistemasolar,* que viene definida por las etiquetas de apertura y cierre *<sistemasolar> y </sistemasolar>*. Entre estas dos etiquetas se encuentra la definición de cada planeta cuyo nombre se define con un *atributo* en la etiqueta principal *<planeta nombre=”<nombre>”></planeta>* y la descripción de cada planeta se encuentra ubicada dentro de cada bloque siguiendo el mismo patrón. Esta unidad no pretende profundizar en los conocimientos de XML, que ya se habrán visto en el módulo de Lenguaje de Marcas del curso primero, pero cabe decir que este tipo de ficheros se utilizan tanto para guardar información como para ser intercambiados entre diferentes sistemas de bases de datos o aplicaciones. De ahí su importancia.

Python provee herramientas para trabajar con este tipo de ficheros. La librería estándar es **xml.etree.ElementTree (ET).**

Cuenta con una serie de funciones, de las que las más utilizadas son:

- **ET.parse(source, parser=None):** devuelve un objeto de tipo *ElementTree* extraído del parámetro source. Este puede ser la ruta de un fichero o un objeto de tipo *file.*
- **ET.SubElement(parent, tag, attrib={}, **extra):** permite crear subelementos para añadir a *parent*; *tag* es el nombre que se le da al elemento y *attrib* se utilizará para definir los atributos que deberá tener. Lo parámetros clave y valor de *extra* se añadirán como atributos extra.
- **ET.fromstring(text, parser=None):** permite parsear la cadena de caracteres que se pasa como *text* en un objeto tipo *Element*
- **Class ET.Element(tag, attrib={}, **extra):** representa un elemento XML que contiene el nombre de la etiqueta proporcionado por *tag* y los atributos añadidos tanto en *attrib* como en *extra* haciendo uso de variables clave-valor. Vemos los atributos y las funciones más comunes con los que cuenta el elemento Element que obtengamos:
  - **attrib**: es un diccionario con todos los atributos del elemento.
  - **tag**: es el nombre que identifica al elemento.
  - **get(key, default=None):** permite devolver el valor del atributo *key* del elemento. Si no lo encuentra devuelve el valor asignado a *default.*
  - **find(match, namespaces=None):** devuelve el primer elemento que coincida con *match.* Si match es una ruta, devuelve el primer elemento de esa ruta. Permite utilizar *namespaces* para filtrar la búsqueda al espacio de nombres entre los subelementos.
  - **findall(match, namespaces=None):** similar a *find* pero devolviendo una lista de subelementos.
  - **findtext(match, default=None, namespaces=None):** busca en el elemento y los subelementos hasta encontrar uno igual a *match.* Si lo encuentra devuelve el texto del elemento, si no, el valor asignado a default.
  - **interfind(match, namespaces=None):** similar a findall pero devolviendo un iterador.
  - **intertext():** crea un iterador que devuelve todos los textos internos del elemento y los subelementos.
- **Class ET.ElementTree(element=None, file=None):** hace referencia al árbol completo tomando como referencia *element.* Lo normal es que se utilice el elemento raíz pero si se toma otro elemento distinto, se devolverá un nuevo árbol que toma como elemento raíz ese elemento.
  - **find, findall, findtext, iterfind**: iguales a las funciones de *Element* pero con un arbol completo.
  - **getroot():** devuelve el elemento raíz.
  - **write(file, encoding="utf-8", xml_declaration=None, default_namespace=None, method="xml", *, short_empty_elements=True):** permite la escritura del árbol asociado en el fichero XML *file.* En *method* se puede establecer si se trata de xml, text o html.

En el siguiente ejemplo, a partir del fichero **planetas.xml**, se desarrolla una función que extrae la información, itera por cada nodo y crea objetos de tipo Planeta. A continuación, con los Planetas ya creados, exportamos el contenido a un nuevo fichero **xml** añadiendo el elemento **gravedad** a cada planeta:

manipular_planetas.py

```
import xml.etree.ElementTree as ET

from ficherodelimitado import Planeta

def extraer_planetas_de_xml(fichero_planetas):
    planetas = []
    xml = ET.parse(fichero_planetas)
    planetas_xml = xml.findall('planeta')
    for planeta_xml in planetas_xml:
        planetas.append(extraer_planeta_de_xml(planeta_xml))
    return planetas


def extraer_planeta_de_xml(planeta_xml):
    nombre = planeta_xml.attrib['nombre']
    masa = planeta_xml.find('masa').text
    radio = planeta_xml.find('radio').text
    return Planeta(nombre, masa, radio)


def escribir_planetas(planetas, fichero_salida):
    raiz = ET.Element('sistemasolar-completo')  # raiz del arbol final
    for planeta in planetas:
        planeta_xml = ET.Element('planeta', attrib=dict(nombre=planeta.nombre))
        masa_xml = ET.Element('masa')  # agregando cada elemento de cada planeta
        masa_xml.text = str(planeta.masa)
        radio_xml = ET.Element('radio', attrib=dict(unidad='km'))
        radio_xml.text = str(planeta.radio)
        gravedad_xml = ET.Element('gravedad')
        gravedad_xml.text = str(planeta.gravedad)
        planeta_xml.extend([masa_xml, radio_xml, gravedad_xml])
        raiz.append(planeta_xml)
    arbol = ET.ElementTree(raiz)  # creando un objeto arbol para poder guardar
    arbol.write(fichero_salida)  # guardando el arbol completo en un fichero
```

```
if __name__ == "__main__":
    fichero_xml = 'planetas.xml'
    planetas = extraer_planetas_de_xml(fichero_xml)

    fichero_xml_salida = 'planetas_completos.xml'
    escribir_planetas(planetas, fichero_xml_salida)
```

La escritura es algo más compleja que la lectura, dado que hay que definir cada elemento del árbol e ir construyendo nodo a nodo, pero una vez que se automatiza, tiene mucho potencial.

2.5. Trabajar con ficheros JSON.

El formato JSON es muy utilizado en aplicaciones web dado que permite intercambiar datos entre aplicaciones (sistemas) de forma rápida y con un número de caracteres extra muy pequeño. Su sintaxis es muy simple y sólo puede contener los siguientes elementos:

- **Números**, tanto positivos como negativos. Se utiliza el carácter punto (.) para separar la parte entera de la decimal.
- **Cadenas de caracteres**, con longitud variable, siempre entre comillas dobles.
- **Booleanos**, sólo permite el uso de *true* para verdadero y *false* para falso.
- **Vacío**, se utiliza *null,* que sería el equivalente al None de Python
- **Listas**, que en JSON son arrays de longitud variable en los que las variables se separan con comas y se escriben entre corchetes ([ ]).
- **Objetos**, que son la parte principal de JSON. Son pares de clave-valor separados por dos puntos (:), similares a los diccionarios de Python, salvo que no están ordenados como sí lo están en Python.

El siguiente es el fichero JSON de los planetas:

planetas.json

```
[
 {
  "nombre": "Mercurio",
  "masa": 3.303e+23,
  "radio": 2439700
 },
 {
  "nombre": "Venus",
  "masa": 4.869e+24,
```

```
  "radio": 6051000
},
{
  "nombre": "Tierra",
  "masa": 5.976e+24,
  "radio": 6378140
},
{
  "nombre": "Marte",
  "masa": 6.421e+23,
  "radio": 3397000
},
{
  "nombre": "Jupiter",
  "masa": 1.9e+27,
  "radio": 71492000
},
{
  "nombre": "Saturno",
  "masa": 5.688e+26,
  "radio": 60268000
},
{
  "nombre": "Urano",
  "masa": 8.686e+25,
  "radio": 25559000
},
{
  "nombre": "Neptuno",
  "masa": 1.024e+26,
```

```
    "radio": 24746000
  }
]
```

Como se puede comprobar, se necesitan menos caracteres que en XML.

En Python existe una librería que soporta los documentos JSON denominada **json** y que cuenta con los siguientes métodos:

- **json.load(file, *, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw)**: permite deserializar el contenido del fichero pasado por parámetro a objetos en Python. Los parámetros son:
  - **file**, fichero de texto o binario con el documento JSON.
  - **cls**, permite establecer un decodificador propio y no el que está por defecto.
  - **object_hook**, permite definir una función para convertir los literales del fichero en objetos.
  - **object_pairs_hook,** similar al anterior pero los literales se pasan como lista de pares.
  - **parse_float**, permite definir una función para parsear los objetos tipo float. Por defecto será float(cadena).
  - **parse_int**, permite definir una función para parsear los objetos tipo int. Por defecto será int(cadena).
  - **parse_constant,** permite definir una función para parsear constantes.
- **json.loads(cadena, *, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw):** es similar a *load* pero se utiliza con una cadena de caracteres (cadena) en lugar de un fichero.
- **json.dump(obj, fp, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_Keys=False, **kw)**: permite serializar un objeto de Python a un fichero JSON en el fichero *fp.* Los parámetros:
  - **obj**, es el objeto a serializar.
  - **fp**, fichero donde se guardará el contenido serializado.
  - **skipkeys**, si tiene un valor a True, las claves del diccionario que no sean de tipo básico serán omitidas, siendo los tipos básicos (str, int, float, bool y None)
  - **ensure_ascii,** por defecto es True y asegura que todos los caracteres no ASCII serán transformados.
  - **check_circular,** por defecto es True y define que se debe comprobar si hay objetos que se referencian a sí mismos. Si se establece a False puede provocar bloques infinitos.
  - **allow_nan**, su valor por defecto es True para convertir los valores de punto flotante en sus análogos JavaScript.
  - **cls**, permite definir serializadores que hereden de la clase json.JSONDecoder.

- **indent**, permite especificar el carácter de indentado en cada nueva línea. Si se especifica un número representará el número de espacios en blanco para el indentado. Si se utiliza un 0 o una cadena vacía, sólo se hará uso de los saltos de línea. La opción por defecto es None, con la que no se añade ni salto de línea ni indentación.
- **separators**, especifica los separadores como una tupla (separador_de_elemento, separador_de_clave), que por defecto son (‘,’ , ’:’).
- **default**, permite describir una función que devuelve la versión JSON de cualquier valor que no pueda ser convertido automáticamente.
- **sort_keys**, si se especifica a True se devuelven los diccionarios ordenados por sus claves.

- **json.dumps(obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw):** esta función es similar a la anterior pero en lugar de guardarlo en un archivo, se devuelve la cadena de caracteres.
- **class json.JSONDecoder(*, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, strict=True, object_pairs_hook=None):** clase que especifica el decodificador para serializar por defecto. Convierte variables JSON a Python, normalmente usando una **clase** que hereda de *JSONDecoder.*
- **class json.JSONEncoder(*, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, sort_keys=False, indent=None, separators=None, default=None):** especifica el codificador para llevar a cabo la serialización por defecto, convirtiendo variables Python en formato JSON y normalmente utilizando una clase propia que hereda de JSONEncoder.

La librería *json* realiza la conversión de JSON a Python según la siguiente tabla:

Tipo en JSON	Tipo en Python
`object {}`	`dict`
`array []`	`list, tuple`
`string`	`str`
`number` (int)	`int`
`number` (float)	`float`
`true`	`True`
`false`	`False`
`null`	`None`

Por lo tanto, es necesario implementar conversores de JSON a Python y viceversa, propios, cuando se trabaja con objetos propios u objetos del núcleo de Python no incluidos en la tabla anterior. (time, decimals…)

Vamos a ver todo esto con un ejemplo completo:

Crea el fichero **planetas.json** y guárdalo en la misma ruta que el ejercicio.

Crear el fichero **ficherosJSON.py** con el siguiente código:

```
import json
from dataclasses import dataclass
# definiendo la clase Planeta con el decorador @dataclass para simplificar la
creación de clases
# importando el módulo json para trabajar con archivos JSON
# en la propia clase definimos el decorador @property para definir un método como
propiedad que calcula la gravedad
@dataclass
class Planeta:
    nombre: str
    masa: float
    radio: float
    # definimos el método __post_init__ para convertir los atributos a float después
de la inicialización
    # esto es útil si los datos se leen de un archivo y vienen como cadenas
    def __post_init__(self):
        self.radio = float(self.radio)
        self.masa = float(self.masa)
    # propiedad para calcular la gravedad del planeta
    # usando el decorador @property para definir un método como propiedad
    @property
    def gravedad(self):
        g_constante = 6.673e-11 # constante de gravitación universal
        # calculando la gravedad usando la fórmula g = constante * masa / radio al
cuadrado
        return g_constante * self.masa / (self.radio ** 2)

# definiendo la función como_planetas que convierte un diccionario en una instancia
de la clase Planeta
# esta función se utiliza como un "object_hook" al cargar (leer) el JSON con el
método json.load
# el "object_hook" permite personalizar la forma en que se convierten los objetos
JSON en instancias de clases
def como_planetas(dct):
    return Planeta(dct['nombre'], dct['masa'], dct['radio'])

# definiendo la clase PlanetaEncoder que hereda de json.JSONEncoder tal y como se
describe en la definición de la función
#JSONEncoder
# esta clase se utiliza para personalizar la forma en que se convierten las
instancias de la clase Planeta a JSON
```

```
# al sobrescribir el método default, podemos definir cómo se deben serializar los
objetos de la clase Planeta
class PlanetaEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Planeta):
            return dict(nombre=obj.nombre, masa=obj.masa, radio=int(obj.radio),
gravedad=obj.gravedad)
        return json.JSONEncoder.default(self, obj)


if __name__ == '__main__':
    with open('planetas.json', 'r') as fr:
        # cargando el archivo JSON y convirtiendo cada diccionario en una instancia
de la clase Planeta
        # usando el método json.load con el "object_hook" como_planetas para
convertir los diccionarios en instancias de la clase Planeta
        planetas = json.load(fr, object_hook=como_planetas)

    # creando un nuevo archivo JSON con los planetas y sus respectivas propiedades
    # usando el método json.dump para escribir los planetas en el archivo JSON con
indentación (indent) de 4 espacios
    with open('planetas_con_gravedad.json', 'w') as fw:
        json.dump(planetas, fw, cls=PlanetaEncoder, indent=4)
```

Prueba el código y observa los comentarios para entender correctamente su funcionamiento.

Python cuenta con otras librerías para el tratamiento de ficheros HTML, YAML.

En Python no existe una librería específica para leer datos en binario, pero sí que suele haber soporte para cada formato mediante librerías de terceros.

Por último y tal y como hemos visto en algún ejemplo, existe librerías de terceros que permiten trabajar con ficheros en diferentes formatos y simplificar las interfaces para acceder, modificar y guardar esos ficheros. Pero es algo que no trataremos dado que sale del núcleo de Python. Sí mencionamos que las dos librerías más importantes para estos desarrollos son *tablib* (*https://tablib.readthedocs.io/en/stable/*) y *pandas* (*https://pandas.pydata.org/*)